

Statistical Learning Internship Presentation

Rishabh Shukla

BT23MA002

NIT Mizoram

IITM Summer Internship (May–July 2025)

Under : Prof. Neelesh Shankar Upadhye

July 14, 2025



Table of Contents

1. Presentation Objective	3
2. Dataset Description	4–5
3. Regression Models	
3.1 Linear Regression	6–7
3.2 Regularization Techniques	
3.2.1 Ridge Regression	8–9
3.2.2 Lasso Regression	10–11
3.3 Regression Tree	12–13
3.4 Random Forest	14–15
3.5 XGBoost	16–17
3.6 Bayesian Linear Regression	18–19
4. Evaluation and Comparison	20–21
5. Conclusion	22
6. References	23

Presentation Objective

- Used Diamonds dataset (12k rows)
- Target: Price (continuous)
- Features: Numerical + few categorical
- EDA: Missing Value Handling, encoding, scaling
- Models:
 - OLS, Ridge, Lasso
 - Tree, RF, XGBoost
 - Bayesian Linear Regression
- Implemented from scratch
- Used Train/Test R^2 to evaluate
- Plotted predicted vs actual
- Focus: Accuracy + Interpretability
- Final best model chosen after comparison

Dataset Description - **Diamonds Dataset**

- Real-world dataset of **53k diamonds** with price information
- **Source:** Dataset taken from Kaggle
- **Target:** price (continuous); aim is to predict fair value of Diamond
- **Predictors:**
 - Numerical: carat, depth, table, x, y, z
 - Categorical: cut, color, clarity (converted via one-hot encoding)
- **Data Cleaning:**
 - Filtering Categorical Variable **cut** into **Premium** and **Ideal**, **Color** into **E** and **F**
 - Dropped clarity due to low correlation with price
- Final processed data: **12,000 rows** and **8 core predictors**
- Use case: Price estimation in diamond retail industry

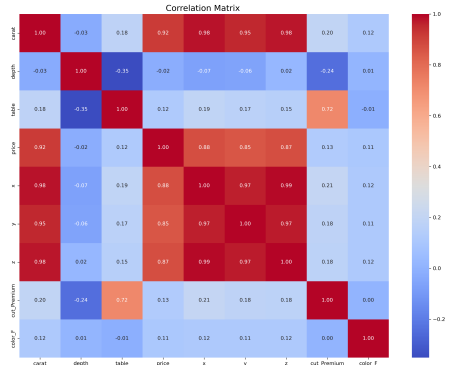
EDA Analysis on Dataset

- Final dataset after preprocessing: **12,397 rows, 9 columns**
- No missing values after cleaning
- Strong correlation with price:
 - carat (0.91), x (0.87), z (0.87), y (0.85)
 - cut_Premium, table, color_F show weak but relevant signals
 - depth has negligible correlation with price
- Top 5 Rows of Dataset and Correlation Matrix :

Top 5 Rows of Datasets After EDA Processing :

	carat	depth	table	price	x	y	z	cut_Premium	color_F
0	0.23	61.5	55.0	326	3.95	3.98	2.43	0	0
1	0.21	59.8	61.0	326	3.89	3.84	2.31	1	0
12	0.22	60.4	61.0	342	3.88	3.84	2.33	1	1
14	0.20	60.2	62.0	345	3.79	3.75	2.27	1	0
15	0.32	60.9	58.0	345	4.38	4.42	2.68	1	0

Top 5 Rows of Final Dataset



Correlation Matrix with Price

Linear Regression

- Linear Regression models the relationship between a continuous target variable y and predictors X using a linear function.
- The model assumes the form:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$$

where $\beta_0, \beta_1, \dots, \beta_p$ are coefficients and ε is the error term.

- In matrix form: $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$
- OLS estimates coefficients by minimizing the RSS:

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

- Prediction equation: $\hat{y} = \mathbf{X}\hat{\boldsymbol{\beta}}$

Pros:

- Simple, fast, and easy to interpret
- Works well when the relationship is linear

Cons:

- Sensitive to multicollinearity and outliers
- Assumes homoscedasticity and normal errors

Linear Regression Implementation on Dataset

- **Estimated Intercept and Coefficients:**

$$\hat{y} = \beta_0 + \beta_1 * \text{carat} + \beta_2 * \text{cut_Premium} + \beta_3 * \text{color_F} + \beta_4 * \text{depth} + \beta_5 * \text{table}$$

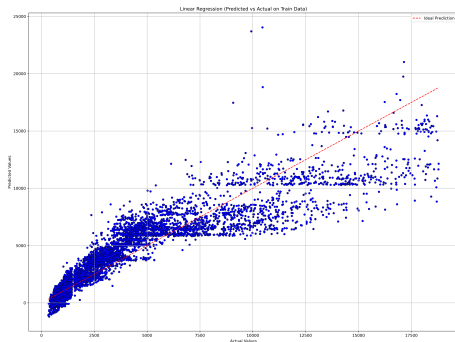
$$\beta_0 = -114.41 \quad \beta_1 = 8830.53 \quad \beta_2 = -404.28$$

$$\beta_3 = -12.79 \quad \beta_4 = -17.02 \quad \beta_5 = -22.98$$

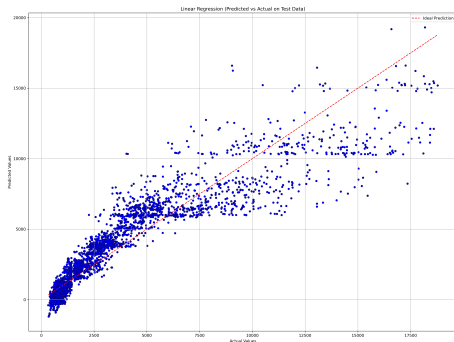
- **Model Performance (R^2 Score):**

- Train $R^2 = 0.84$

- Test $R^2 = 0.85$



Trained Set: Predicted vs Actual



Test Set: Predicted vs Actual

Ridge Regression

- Ridge adds a **penalty on large coefficients** to reduce overfitting.
- Helps when predictors are **correlated** or when p is large.
- Objective function:

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

- λ is a **tuning parameter** controlling penalty strength:
 - $\lambda = 0 \rightarrow$ equivalent to OLS
 - $\lambda > 0 \rightarrow$ coefficients shrink toward 0
 - $\lambda \rightarrow \infty \rightarrow$ all $\beta_j \rightarrow 0$
- Uses **L2 norm** penalty (squared):

$$\text{Penalty} = \lambda(\beta_1^2 + \beta_2^2 + \dots + \beta_p^2)$$

- Requires **centering predictors**:

$$x_j^{\text{centered}} = x_j - \bar{x}_j$$

- Intercept β_0 is excluded from the penalty term.
- **Pros:**
 - Reduces overfitting and model variance
 - Useful when p is large or predictors are correlated
- **Cons:**
 - Requires careful tuning of λ
 - Less interpretable when many small non-zero coefficients

Ridge Regression Implementation on Dataset

- **Best λ (Regularization Strength): 10.2353**

- **Estimated Intercept and Coefficients:**

$$\hat{y} = \beta_0 + \beta_1 * \text{carat} + \beta_2 * \text{depth} + \beta_3 * \text{table} + \beta_4 * \text{x} + \beta_5 * \text{y} + \beta_6 * \text{z} \\ + \beta_7 * \text{cut_Premium} + \beta_8 * \text{color_F}$$

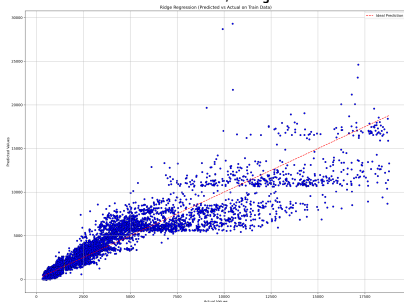
$$\beta_0 = 16163.53 \quad \beta_1 = 14779.55 \quad \beta_2 = -122.01$$

$$\beta_3 = -39.73 \quad \beta_4 = -2335.49 \quad \beta_5 = -52.15$$

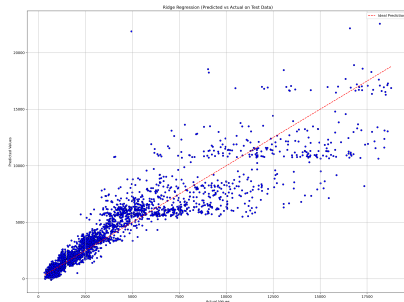
$$\beta_6 = 44.02 \quad \beta_7 = -334.57 \quad \beta_8 = -0.69$$

- **Model Performance:**

- Train $R^2 = 0.86$, Adjusted $R^2 = 0.86$
- Test $R^2 = 0.86$, Adjusted $R^2 = 0.86$



Trained Set: Predicted vs Actual



Test Set: Predicted vs Actual

Lasso Regression

- Lasso stands for **Least Absolute Shrinkage and Selection Operator**.
- Similar to Ridge but uses the **L1 penalty** instead of L2:

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

- Unlike Ridge, Lasso can shrink some coefficients **exactly to 0**:
 - Enables **automatic feature selection**
 - Produces **sparse, interpretable models**
- λ controls regularization strength:
 - $\lambda = 0 \rightarrow$ Lasso = OLS (all variables used)
 - λ increases $\rightarrow$ more coefficients are set to 0
- **L1 vs L2** norms:

$$\text{L1 norm: } |\beta_1| + |\beta_2| + \dots + |\beta_p|, \quad \text{L2 norm: } \sqrt{\sum \beta_j^2}$$

- Geometry of L1 penalty encourages coefficients to hit exact zero
- Best λ is chosen via **cross-validation**
- **Pros:**
 - Performs feature selection by eliminating irrelevant variables
 - Reduces overfitting and simplifies the model
 - Useful when p is large but only a few predictors matter
- **Cons:**
 - Can be unstable with highly correlated predictors
 - May discard useful variables under strong regularization
 - Solution is biased due to absolute penalty

Lasso Regression Implementation on Dataset

- Same $\hat{y}$ equation and features for Lasso.
- **Best λ (Regularization Strength): 0.6280**
- **Estimated Intercept and Coefficients:**

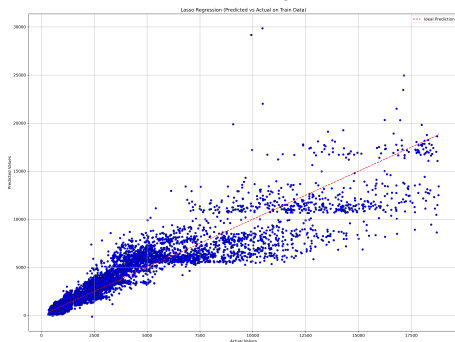
$$\beta_0 = 17987.73 \quad \beta_1 = 15341.67 \quad \beta_2 = -137.84$$

$$\beta_3 = -41.00 \quad \beta_4 = -3062.74 \quad \beta_5 = 381.16$$

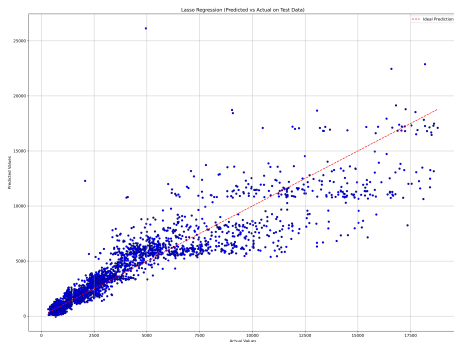
$$\beta_6 = 176.08 \quad \beta_7 = -312.05 \quad \beta_8 = -0.50$$

- **Model Performance:**

- Train $R^2 = 0.86$, Adjusted $R^2 = 0.86$
- Test $R^2 = 0.86$, Adjusted $R^2 = 0.86$



Trained Set: Predicted vs Actual



Test Set: Predicted vs Actual

Regression Tree

- A **Regression Tree** splits the predictor space into non-overlapping regions $R_1, R_2, \dots, R_J$. For a region R_j , the prediction is the mean of the target variable:

$$\hat{y}_{R_j} = \frac{1}{|R_j|} \sum_{x_i \in R_j} y_i$$

- Built using a **top-down, greedy** approach called **Recursive Binary Splitting**.
- At each node, choose predictor X_j and split point s that minimizes the Residual Sum of Squares (RSS):

$$RSS = \sum_{i \in R_1(j,s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i \in R_2(j,s)} (y_i - \bar{y}_{R_2})^2$$

- Apply **Cost-Complexity Pruning** to regularize tree size:

$$C_\alpha(T) = RSS(T) + \alpha \cdot |T|$$

where $|T|$ is number of terminal nodes and $\alpha \geq 0$ is the penalty.

- Best α selected via **K-fold cross-validation**.
- Performs well when:
 - Data is **non-linear** or contains **interactions**
 - Predictors are a mix of **categorical and continuous**

Pros:

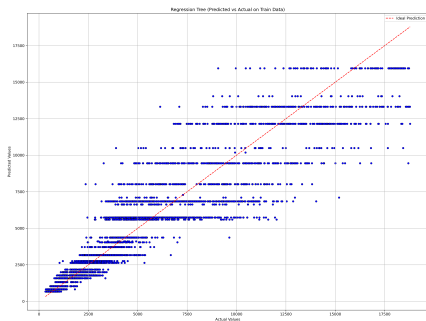
- Easy to interpret and visualize
- No need to scale features
- Handles non-linear relationships

Cons:

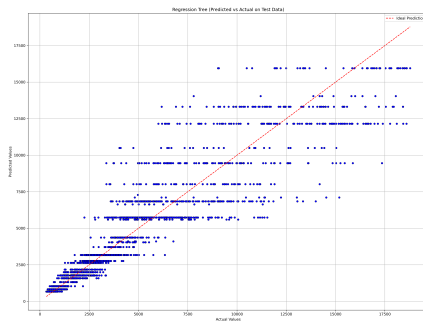
- Prone to overfitting without pruning
- Sensitive to data variations (unstable)
- Less accurate than ensemble models

Regression Tree Implementation on Dataset

- **Best Pruning Parameter α (via Cross-Validation):** 0.0001
- **Tree Structure:**
 - Tree Depth: 5
 - Internal Nodes: 29
 - Number of Leaves: 30
- **Model Performance:**
 - Train $R^2 = 0.8899$
 - Test $R^2 = 0.8766$



Trained Set: Predicted vs Actual



Test Set: Predicted vs Actual

Random Forest

- **Random Forest** is an **ensemble learning method** that builds multiple decision trees and combines their outputs to improve performance and reduce overfitting.
- It follows two key techniques:
 - **Bootstrap Sampling**: Each tree is trained on a random sample (with replacement) of the training data.
 - **Feature Subsampling**: At each split, a random subset of features is considered.
- For regression tasks, prediction is the average of all tree outputs:

$$\hat{y}_{\text{RF}} = \frac{1}{T} \sum_{t=1}^T \hat{y}^{(t)}$$

where T is the number of trees and $\hat{y}^{(t)}$ is the prediction from tree t .

- Each tree is **grown deep without pruning**, reducing bias but increasing variance — which is then reduced by averaging across trees.

Pros:

- Reduces overfitting via averaging
- Handles high variance and noise well
- Works well with non-linear and correlated features
- Requires little data preprocessing

Cons:

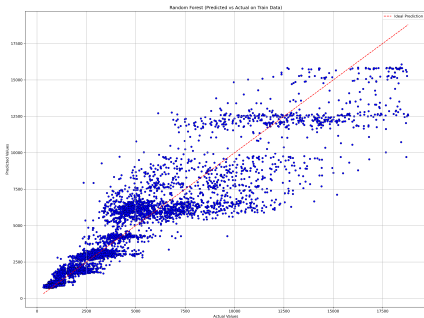
- Less interpretable than a single tree
- Computationally expensive with many trees
- May overfit if too many deep trees without tuning

Random Forest Implementation on Dataset

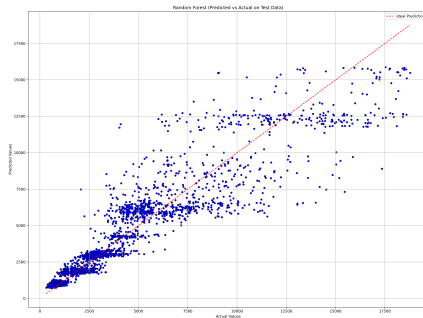
- **Model Performance:**

- Train $R^2 = 0.8898$
- Test $R^2 = 0.8817$

- **Plot of Predicted vs Actual**



Trained Set: Predicted vs Actual



Test Set: Predicted vs Actual

XGBoost (Extreme Gradient Boosting)

- **XGBoost** is a boosting-based ensemble algorithm that builds trees sequentially, each correcting the residual errors of the previous ones.
- Prediction is built step-by-step:

$$\hat{y}_i = \sum_{m=1}^M \eta f_m(x_i)$$

where η is the learning rate and f_m is the m^{th} regression tree.

- The objective function includes a regularization term to penalize complexity:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t)}) + \sum_{m=1}^M \Omega(f_m)$$

with $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$

- Learning proceeds by:
 - Computing residuals from current predictions
 - Fitting a new tree to the residuals
 - Updating the prediction using the learning rate

Pros:

- High predictive accuracy
- Captures complex interactions
- Built-in regularization reduces overfitting

Cons:

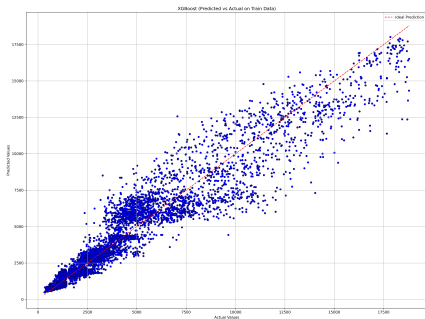
- Sensitive to hyperparameter tuning
- Slower to train than simpler models
- Can overfit noisy data

XGBoost Implementation on Dataset

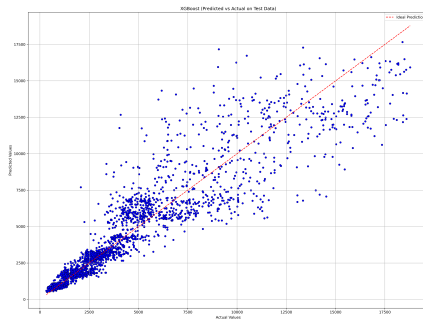
- **Model Performance:**

- Train $R^2 = 0.9314$
- Test $R^2 = 0.8786$

- **Plot of Predicted vs Actual**



Trained Set: Predicted vs Actual



Test Set: Predicted vs Actual

Bayesian Linear Regression

- **Bayesian Linear Regression** treats the model parameters $\mathbf{w}$ as random variables and computes a full posterior distribution over them, given the data.
- Prior belief over parameters:

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$$

- Likelihood of the observed data:

$$p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}, \sigma^2) = \mathcal{N}(\mathbf{y} \mid \mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I})$$

- Posterior over weights (Bayes rule):

$$p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \mathcal{N}(\mathbf{w} \mid \boldsymbol{\mu}_n, \boldsymbol{\Sigma}_n)$$

where:

$$\boldsymbol{\Sigma}_n = \left(\boldsymbol{\Sigma}_0^{-1} + \frac{1}{\sigma^2} \mathbf{X}^\top \mathbf{X} \right)^{-1}, \quad \boldsymbol{\mu}_n = \boldsymbol{\Sigma}_n \left(\boldsymbol{\Sigma}_0^{-1} \boldsymbol{\mu}_0 + \frac{1}{\sigma^2} \mathbf{X}^\top \mathbf{y} \right)$$

- **Posterior predictive distribution** for a new input $\mathbf{x}_*$:

$$p(y_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}) = \mathcal{N}(y_* \mid \mathbf{x}_*^\top \boldsymbol{\mu}_n, \mathbf{x}_*^\top \boldsymbol{\Sigma}_n \mathbf{x}_* + \sigma^2)$$

- If prior is $\mathcal{N}(0, \tau^2 \mathbf{I})$, and $\lambda = \sigma^2 / \tau^2$, then the posterior mean matches Ridge Regression solution:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

Pros:

Cons:

- | | |
|--|---|
| • Provides full uncertainty estimation | • Computationally intensive for large n |
| • Naturally regularized by the prior | • Requires choice of prior |

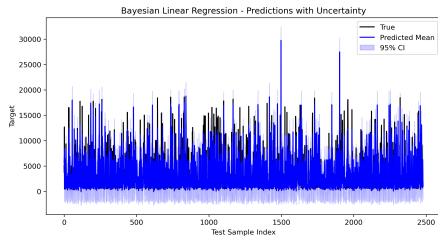
Bayesian Linear Regression on Dataset

• Posterior Mean Coefficients with Uncertainty:

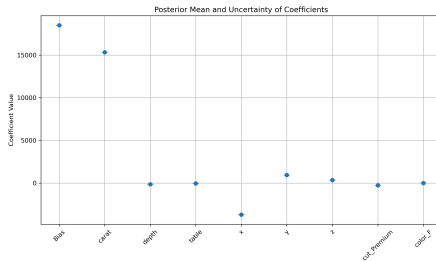
Bias (Intercept): 18492.6351 ± 0.0101
carat: 15327.7246 ± 0.1425
depth: -149.4332 ± 0.0164
table: -38.5196 ± 0.0080
x: -3714.5177 ± 0.2282
y: 939.6361 ± 0.2280
z: 347.5968 ± 0.1948
cut_Premium: -271.9744 ± 0.0312
color_F: 4.2379 ± 0.0202

• Model Performance:

- Train R^2 : **0.8627**
- Test R^2 : **0.8251**



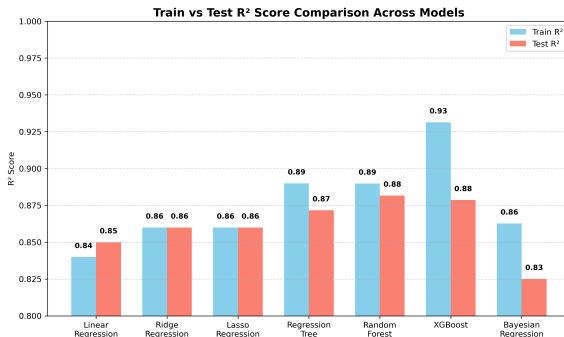
Prediction Uncertainty with 95% Confidence Interval



Posterior Mean and Uncertainty of Coefficients

Model Comparison – Accuracy-Based Evaluation

- Below plot compares Train vs Test R^2 Score for all models.
- Best Accuracy:** **XGBoost** achieved highest train R^2 (0.93) and strong test R^2 (0.88).
- Most Consistent:** Ridge, Lasso, and Random Forest show high and stable performance.
- Bayesian Regression** shows slightly lower accuracy but offers **uncertainty quantification**.



Model Insights – Strengths and Limitations

- **Linear Regression:**

- Works well for **linear relationships**, interpretable and fast.
- Fails when multicollinearity or non-linearity exists.
- Improved by Ridge/Lasso for regularization and feature control.

- **Ridge & Lasso Regression:**

- Ridge reduces overfitting by shrinking coefficients.
- Lasso performs **feature selection** by zeroing weak predictors.
- Both maintain high R^2 with **better generalization**.

- **Regression Tree:**

- Captures **non-linearity** and interactions easily.
- May overfit if not pruned.
- Random Forest stabilizes variance and boosts robustness.

- **Random Forest:**

- Averaging across trees reduces variance.
- Handles mixed-type data well.
- Sometime Random Forest Improve by XGBoost in form of Accuracy.

- **XGBoost:**

- Achieves **best accuracy** via gradient boosting.
- Can overfit with small datasets.
- Less interpretable but highly tunable and powerful.

- **Bayesian Linear Regression:**

- Offers **uncertainty estimation** and probabilistic interpretation.
- Performs similar to Ridge but with full posterior knowledge.
- Ideal when quantifying confidence is essential.

- Applied a wide range of regression models on the **Diamonds Dataset** to predict **price**.
- **XGBoost** emerged as the best performer in terms of predictive **accuracy** (**Train** $R^2 = 0.93$, **Test** $R^2 = 0.88$).
- **Ridge and Lasso** offered strong, interpretable models with excellent generalization and automatic feature control.
- **Bayesian Linear Regression** provided valuable *uncertainty estimation* along with solid performance.
- Comprehensive comparison enabled a balance between **accuracy, interpretability, and uncertainty quantification**.
- Models were implemented from scratch with detailed understanding of mathematical formulation and optimization.
- Future scope includes exploring more advanced techniques like **deep learning** and **neural networks** for further improvement.

References

Internship under Prof. Neelesh Shankar Upadhye — IITM, Summer 2025

- **James, G., Witten, D., Hastie, T., Tibshirani, R.**
An Introduction to Statistical Learning in Python (ISLP)
- **Kevin P. Murphy**
Probabilistic Machine Learning: Foundations and Advanced Topics (Vol. 1 & 2)

Thank You!

Questions and Discussions are Welcomed

Acknowledgements:

- Director: **Prof. S. Sundar** (NIT Mizoram)
- Internship Guide: **Prof. Neelesh Shankar Upadhye**
- PhD Mentor: **Mr. Raju Chaudhary**